

MC714 — Sistemas Distribuídos

Trabalho 2

João Guilherme Ferreira Pedra — RA 248349

1. Introdução

Em sistemas distribuídos, temos algumas problemáticas importantes para refletir na transação entre processos: ordenação de eventos que ocorrem em processos diferentes, sem um relógio físico global comum; garantir que apenas um processo por vez acesse um recurso compartilhado; e eleger um processo coordenador dentre vários, mesmo sem que os processos conheçam previamente o estado uns dos outros.

Este trabalho implementa um algoritmo para cada um desses problemas: Relógio Lógico de Lamport, Token Ring e Eleição em Anel integrados em um único sistema de nós que se comunicam por troca de mensagens reais via sockets TCP, executando em containers Docker distintos.

2. Algoritmos Escolhidos

2.1 Relógio Lógico de Lamport

A partir do algoritmo descrito em [1], consegui utilizar a lógica central da classe LamportClock, com os métodos `increment()` e `update()` (este último aplicando $\max(\text{local}, \text{recebido}) + 1$), além do uso de um socket TCP simples para transmitir o timestamp entre processos. Entretanto, dado o escopo diferente do projeto, foi removida a interface de terminal do projeto original; a classe foi integrada ao loop principal do nó, que já recebe seus parâmetros (id, próximo nó do anel, portas) por linha de comando em vez de input interativo.

2.2 Exclusão Mútua — Token Ring

No algoritmo de token ring, utilizei a fonte consultada em [2], da qual foram aproveitados a lógica de posse do token (`has_token`) e a ideia geral da sequência de operações `requestToken` → `checkTokenAndEnterCS` → `enterCriticalSection` → `passToken`. O código original, no entanto, mantém uma fila de pedidos por processo,

já que o token é enviado diretamente para quem o solicitou, percorrendo o anel até encontrar quem está de posse dele (`next.requestToken(...)`). Como nesta implementação o token circula apenas em um sentido fixo a fila foi removida: um nó entra na seção crítica assim que recebe o token e possui um pedido pendente, repassando-o em seguida.

Além disso, a passagem do token deixou de ser uma chamada direta de método entre objetos na mesma JVM (`next.setToken(...)`) e passou a ser uma mensagem TOKEN enviada de fato pelo socket ao próximo nó do anel, já que os processos rodam em containers separados. O código também foi convertido de Java para Python.

2.3 Eleição de Líder

A ideia central do algoritmo [3] seria que cada processo repassa o maior id visto, o id que "dá a volta" completa se torna coordenador, e uma mensagem de coordenador circula avisando todos os processos. Porém, há uma simulação de todo o processo dentro de um único script Python, percorrendo uma lista `self.processes` e usando `print()` para representar a troca de mensagens. Nesta implementação a mensagem ELECTION/ELECTED é enviada por socket entre processos distintos, e a detecção de "a mensagem voltou ao remetente original" é feita comparando o id do candidato recebido com o próprio id do nó.

3. Arquitetura da Implementação

O sistema é composto por N nós organizados em topologia de anel. Cada nó é um processo Python independente, executado em um container Docker próprio, que:

- Escuta conexões TCP de entrada em uma porta fixa (o nó anterior do anel) [4];
- Mantém uma conexão TCP de saída para o próximo nó do anel [4];
- Troca mensagens no formato JSON, delimitadas por quebra de linha (`\n`), contendo tipo, remetente, dado e timestamp lógico [5];
- Executa em paralelo as três lógicas: relógio de Lamport [1], exclusão mútua por token ring [2] e eleição em anel [3].

Os containers são orquestrados via Docker Compose, em uma rede bridge dedicada (`app_network`) [6], o que permite que cada nó resolva o próximo nó do anel

pelo nome do serviço (ex.: node1, node2, ...) em vez de endereço IP fixo.

3.1 Diagrama de Componentes (descrição textual)

node0 → node1 → node2 → node3 → node4 → node0 (anel fechado).

Cada seta representa uma conexão TCP unidirecional estabelecida na inicialização. Mensagens ELECTION, ELECTED, TOKEN, TOKEN_REQUEST e LAMPORT_EVENTO trafegam nesse sentido único.

3.2 Arquitetura

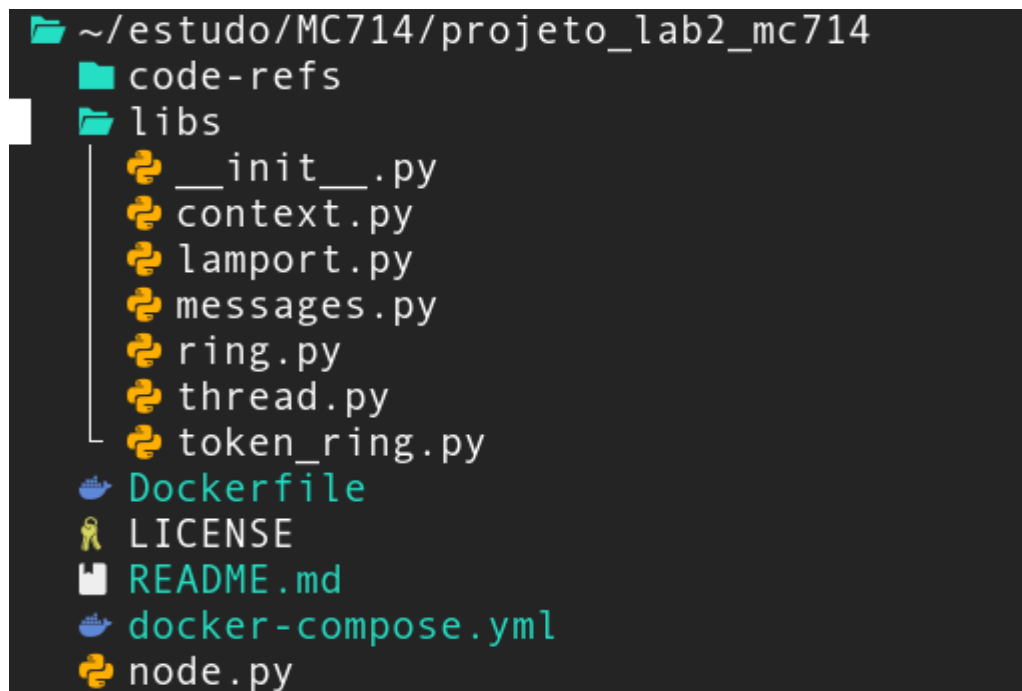


Fig 1: Estrutura de arquivos.

4. Testes e Experimentos

Os testes a seguir verificam as propriedades fundamentais de cada algoritmo, incluindo comportamento em situações normais e em cenários de limite/falha.

#	Algoritmo	Teste	Comando(s)	Resultado esperado
T1.1	Lamport	Causalidade respeitada	<code>docker compose up --build / docker compose logs node0 node1</code>	Timestamp do evento recebido é sempre maior que o do evento causador
T1.2	Lamport	Mensagem atrasada	Inserir <code>time.sleep</code> antes de um envio; rodar normalmente	Relógio ainda converge corretamente via $\max(\text{local}, \text{recebido}) + 1$
T1.3	Lamport	Eventos concorrentes (sem relação causal)	<code>docker-compose logs node2 node4</code>	Podem ocorrer timestamps iguais/fora de ordem real — limitação conhecida do algoritmo
T2.1	Token Ring	Segurança e vivacidade	<code>docker-compose logs grep ENTERED EXIT</code>	Nunca dois nós em seção crítica simultaneamente; todo pedido é eventualmente atendido
T2.2	Token Ring	Nó cai antes de repassar o token	<code>Docker-compose stop nodeX</code>	Anel trava: token nunca mais circula para os nós após X

T2.3	Token Ring	Nó cai durante a seção crítica	Docker-compose stop <container em SC>	Token se perde; nenhum outro nó consegue entrar na SC (deadlock)
T3.1	Eleição	Convergência	docker compose logs grep "LIDER CONFIRMADO"	Todos os nós concordam no mesmo líder (maior ID ativo)
T3.2	Eleição	Duas eleições simultâneas	Dois nós chamam start() quase ao mesmo tempo	Anel converge para um único líder mesmo com concorrência
T3.3	Eleição	Líder cai antes de anunciar	docker-compose stop <líder> logo após "SOU O LIDER"	Nenhum ELECTED circula; anel fica sem líder confirmado
T3.4	Eleição	Nó cai durante repasse do candidato	docker-container stop <nó> durante "encaminhando candidato"	Mensagem nunca completa a volta; eleição trava

6. Resultados

6.1 Teste 1.1

```
node4 | [Listen] Waiting for connection on port 6000
Container node1 Started
node3 | [Listen] Waiting for connection on port 6000
Container node0 Started
node2 | [Listen] Waiting for connection on port 6000
node1 | [Listen] Waiting for connection on port 6000
node0 | [Listen] Waiting for connection on port 6000
node4 | [node 4] connected to node0:6000
node4 | [node 4 | T=1] sent Lamport event
node0 | [Listen] previous node connected
node4 | [Listen] previous node connected
node3 | [node 3] connected to node4:6000
node3 | [Listen] previous node connected
node2 | [node 2] connected to node3:6000
node1 | [node 1] connected to node2:6000
node2 | [Listen] previous node connected
node1 | [Listen] previous node connected
node0 | [node 0] connected to node1:6000
node0 | [node 0 | T=1] started election with candidate 0
node0 | [node 0 | T=2] received Lamport event from node 4
```

Fig 2: Teste 1.1 – Envio e recebimento de um evento de Relógio de Lamport.

O trecho de log mostra a propriedade do relógio de Lamport: o evento de envio no node4 (T=1) precede causalmente o evento de recebimento no node0 (T=2), respeitando a relação happened-before.

6.2 Teste 1.2

```
def emit_periodic_lamport_event(ctx):
    if random.random() < 0.5:
        return
    time.sleep(5) # /T1.2
    ts = ctx.clock.increment()
    ctx.send("LAMPORTEVENT", 0, ts)
    now = datetime.datetime.now().strftime("%H:%M:%S")
    ctx.log(f"sent Lamport event (wall-clock: {now})", ts)
```

Fig 3.1: Teste 1.2 – Adicionado atraso no relógio de Lamport.

```
124 node2 | [node 2 | T=1] sent Lamport event (wall-clock: 00:02:54)
125 node1 | [node 1 | T=1] sent Lamport event (wall-clock: 00:02:54)
126 node4 | [node 4 | T=1] sent Lamport event (wall-clock: 00:02:54)
127 node0 | [node 0 | T=2] sent Lamport event (wall-clock: 00:02:54)
128 node3 | [node 3 | T=1] sent Lamport event (wall-clock: 00:02:55)
129 node2 | [node 2 | T=2] received Lamport event from node 1 00:02:58
130 node1 | [node 1 | T=2] forwarding candidate 1
131 node1 | [node 1 | T=4] received Lamport event from node 0 00:02:58
132 node4 | [node 4 | T=2] received Lamport event from node 3 00:02:58
133 node0 | [node 0 | T=3] received Lamport event from node 4 00:02:58
134 node3 | [node 3 | T=2] received Lamport event from node 2 00:02:59
```

Fig 3.2: Teste 1.2 – Envio e recebimento de um evento de Relógio de Lamport com atraso.

O trecho de log evidencia que, mesmo havendo um atraso de rede de aproximadamente 4 segundos entre o envio (wall-clock 00:02:54) e o recebimento (wall-clock 00:02:58) da mensagem, o relógio lógico de Lamport manteve a relação causal esperada (T=1 no envio, T=2 no recebimento), confirmando que a ordenação lógica dos eventos não depende do tempo real de entrega.

6.3 Teste 2.1

```
node0 | [node 0 | T=22] requested critical section (token ring)
node0 | [node 0 | T=23] ==> ENTERED critical section
node1 | [node 1 | T=17] received Lamport event from node 0 00:28:00
node1 | [node 1 | T=22] leader confirmed: node 4
node1 | [node 1 | T=24] requested critical section (token ring)
node0 | [node 0 | T=24] <== LEFT critical section
node0 | [node 0 | T=25] passing initial TOKEN
node4 | [node 4 | T=21] sent Lamport event (wall-clock: 00:28:01)
node2 | [node 2 | T=19] sent Lamport event (wall-clock: 00:28:04)
node1 | [node 1 | T=26] received the TOKEN
node1 | [node 1 | T=27] ==> ENTERED critical section
node0 | [node 0 | T=26] received Lamport event from node 4 00:28:05
node1 | [node 1 | T=28] <== LEFT critical section
```

Fig 4 – Funcionamento do algoritmo de Token Ring.

O trecho de log evidencia o funcionamento do algoritmo de exclusão mútua por Token Ring após a conclusão da eleição de líder. O nó 0, de posse do token inicial, solicita a seção crítica (T=22), entra (T=23) e sai dela (T=24) antes de repassar o token ao próximo nó do anel (T=25). O nó 1, que já havia solicitado a seção crítica anteriormente (T=24), só recebe o token depois que o nó 0 efetivamente saiu da seção crítica, confirmando a propriedade de segurança. A circulação contínua do token entre os nós também garante a propriedade de vivacidade, já que todo nó que solicitar a seção crítica eventualmente receberá o token e poderá entrar.

6.4 Teste 2.2

```
projeto_lab2_mc714: docker-compose stop node3
[+] stop 1/1
✓ Container node3 Stopped 10.1s
```

Fig 5.1 – Nó 3 é desligado.

```
node1 | [node 1 | T=38] received the TOKEN
node1 | [node 1 | T=39] ==> ENTERED critical section
node1 | [node 1 | T=40] <== LEFT critical section
node1 | [node 1 | T=41] passing TOKEN to next node
node4 | [node 4 | T=36] sent Lamport event (wall-clock: 00:44:50)
node0 | [node 0 | T=38] sent Lamport event (wall-clock: 00:44:50)
node3 | [node 3 | T=33] received Lamport event from node 2 00:44:51
^M[Knode3 exited with code 137
node2 | [node 2 | T=42] received the TOKEN
node2 | [node 2 | T=43] ==> ENTERED critical section
node2 | [node 2 | T=44] <== LEFT critical section
node2 | [node 2 | T=45] passing TOKEN to next node
node1 | [node 1 | T=42] received Lamport event from node 0 00:44:53
node0 | [node 0 | T=39] received Lamport event from node 4 00:44:54
node2 | [node 2 | T=46] sent Lamport event (wall-clock: 00:44:58)
node1 | [node 1 | T=43] sent Lamport event (wall-clock: 00:45:02)
node4 | [node 4 | T=37] sent Lamport event (wall-clock: 00:45:03)
node2 | [node 2 | T=47] received Lamport event from node 1 00:45:06
node0 | [node 0 | T=40] sent Lamport event (wall-clock: 00:45:07)
node1 | [node 1 | T=44] received Lamport event from node 0 00:45:10
```

Fig 5.2 – Funcionamento do algoritmo de Token Ring ao fazer o nó 3 falhar.

O trecho de log evidencia a limitação do algoritmo de Token Ring frente à falha de um nó. O nó 2 passou o token ao nó 3 (passing TOKEN to next node), que foi encerrado imediatamente após (node3 exited with code 137,

sinalizando finalização forçada via `docker stop`). A partir desse ponto, nenhuma nova mensagem `TOKEN` circula pelo anel, e nenhum nó consegue mais entrar em seção crítica. Isso confirma que o token, ao se perder junto com o processo que o detinha, não é regenerado automaticamente, evidenciando a ausência de tolerância a falhas nesse algoritmo em sua forma clássica.

6.5 Teste 2.3

```
projeto_lab2_mc714: docker-compose stop node2
[+] stop 1/1
✓ Container node2 Stopped 10.2s
```

Fig 6.1 – Nó 2 é desligado.

```
node2 | [node 2 | T=35] ==> ENTERED critical section 01:13:29
node3 | [node 3 | T=35] sent Lamport event (wall-clock: 01:13:29)
node3 | [node 3 | T=36] received Lamport event from node 2 01:13:33
node2 exited with code 137
node0 | [node 0 | T=29] sent Lamport event (wall-clock: 01:13:52)
```

Fig 6.2 – Funcionamento do algoritmo de Token Ring ao fazer o nó 2 falhar não passa ticket para próximo nó.

O trecho de log evidencia o encerramento abrupto do nó 2 (`node2 exited with code 137`). Após a queda, nenhuma nova mensagem de `TOKEN` ou de entrada/saída em seção crítica é observada no restante do log, indicando que o token foi perdido junto com o processo e não foi mais recuperado. Isso confirma a mesma limitação identificada no teste T2.2: o algoritmo de Token Ring, em sua forma clássica, não possui mecanismo de regeneração do token após a falha do processo que o detém, resultando em impossibilidade permanente de acesso à seção crítica pelos nós remanescentes.

6.7 Teste 3.1

```
node1 | [node 1 | T=12] forwarding candidate 4
node2 | [node 2 | T=14] forwarding candidate 4
node4 | [node 4 | T=11] received Lamport event from node 3
node0 | [node 0 | T=12] sent Lamport event (wall-clock: 23:57:16)
node3 | [node 3 | T=10] sent Lamport event (wall-clock: 23:57:16)
node1 | [node 1 | T=14] sent Lamport event (wall-clock: 23:57:19)
node4 | [node 4 | T=12] received Lamport event from node 3
node0 | [node 0 | T=13] received Lamport event from node 4
node2 | [node 2 | T=16] sent Lamport event (wall-clock: 23:57:20)
node3 | [node 3 | T=16] forwarding candidate 4
node3 | [node 3 | T=18] received Lamport event from node 2
node1 | [node 1 | T=15] received Lamport event from node 0
node4 | [node 4 | T=18] I AM THE LEADER (id=4), sending ELECTED
node4 | [node 4 | T=20] requested critical section (token ring)
node2 | [node 2 | T=17] received Lamport event from node 1
node0 | [node 0 | T=14] sent Lamport event (wall-clock: 23:57:25)
node1 | [node 1 | T=16] sent Lamport event (wall-clock: 23:57:28)
node2 | [node 2 | T=18] received Lamport event from node 1
node0 | [node 0 | T=20] leader confirmed: node 4
node0 | [node 0 | T=22] registered CS request from node 4
node0 | [node 0 | T=24] requested critical section (token ring)
node0 | [node 0 | T=25] passing initial TOKEN
node3 | [node 3 | T=19] sent Lamport event (wall-clock: 23:57:29)
node1 | [node 1 | T=17] received Lamport event from node 0
node1 | [node 1 | T=22] leader confirmed: node 4
node1 | [node 1 | T=24] registered CS request from node 4
node1 | [node 1 | T=26] registered CS request from node 0
node1 | [node 1 | T=28] received the TOKEN
node1 | [node 1 | T=29] passing TOKEN to next node
node1 | [node 1 | T=30] requested critical section (token ring)
node2 | [node 2 | T=24] leader confirmed: node 4
```

Fig 7 – Funcionamento do algoritmo de eleição.

O trecho de log evidencia o algoritmo de eleição em anel. Após o candidato de maior identificador (4) já ter sido determinado nas rodadas anteriores, a mensagem ELECTION contendo esse valor continua circulando pelo anel — sendo repassada sequencialmente pelos nós 0, 1, 2 e 3 (forwarding candidate 4) sem qualquer alteração, já que nenhum deles possui identificador maior. Quando a mensagem retorna ao nó 4, este reconhece seu próprio identificador como o valor circulado e declara-se líder (I AM THE LEADER), disparando o envio da mensagem ELECTED.

Esse comportamento confirma a propriedade central do algoritmo: a eleição converge corretamente para o processo de maior identificador do anel, e a detecção de que a mensagem completou uma volta completa é feita exclusivamente pela comparação do candidato recebido com o próprio identificador do nó, sem necessidade de qualquer contagem explícita de voltas.

6.8 Teste 3.2

```
151 node0 [node 0 T=8] forwarding candidate 4
152 node0 [node 0 T=10] forwarding candidate 4
153 node0 [node 0 T=12] received Lamport event from node 4 01:53:12
154 node1 [node 1 T=7] received Lamport event from node 0 01:53:12
155 node1 [node 1 T=10] forwarding candidate 4
156 node1 [node 1 T=12] forwarding candidate 4
157 node3 [node 3 T=9] sent Lamport event (wall-clock: 01:53:16)
158 node4 [node 4 T=11] received Lamport event from node 3 01:53:16
159 node2 [node 2 T=9] sent Lamport event (wall-clock: 01:53:17)
160 node3 [node 3 T=10] received Lamport event from node 2 01:53:20
161 node2 [node 2 T=12] forwarding candidate 4
162 node2 [node 2 T=14] forwarding candidate 4
163 node4 [node 4 T=12] sent Lamport event (wall-clock: 01:53:21)
164 node1 [node 1 T=14] sent Lamport event (wall-clock: 01:53:21)
165 node0 [node 0 T=13] received Lamport event from node 4 01:53:24
166 node3 [node 3 T=11] sent Lamport event (wall-clock: 01:53:25)
167 node2 [node 2 T=16] received Lamport event from node 1 01:53:25
168 node4 [node 4 T=13] received Lamport event from node 3 01:53:25
169 node3 [node 3 T=14] forwarding candidate 4
170 node3 [node 3 T=16] forwarding candidate 4
171 node4 [node 4 T=14] sent Lamport event (wall-clock: 01:53:30)
172 node1 [node 1 T=15] sent Lamport event (wall-clock: 01:53:30)
173 node2 [node 2 T=17] received Lamport event from node 1 01:53:33
174 node0 [node 0 T=14] sent Lamport event (wall-clock: 01:53:33)
175 node3 [node 3 T=18] sent Lamport event (wall-clock: 01:53:34)
176 node4 [node 4 T=16] I AM THE LEADER (id=4), sending ELECTED
177 node4 [node 4 T=18] I AM THE LEADER (id=4), sending ELECTED
178 node4 [node 4 T=20] received Lamport event from node 3 01:53:34
179 node4 [node 4 T=21] requested critical section (token ring)
180 node1 [node 1 T=16] received Lamport event from node 0 01:53:34
181 node0 [node 0 T=15] received Lamport event from node 4 01:53:37
182 node0 [node 0 T=18] leader confirmed: node 4
183 node0 [node 0 T=20] leader confirmed: node 4
```

Fig 8 – Funcionamento do algoritmo de eleição com duas eleições ao mesmo tempo.

O trecho de log mostra o comportamento do algoritmo de eleição em anel quando duas eleições são iniciadas simultaneamente (nós 0 e 1, nesta execução). Observa-se que cada nó do anel recebe duas mensagens `ELECTED` (uma originada de cada eleição concorrente), como evidenciado pela duplicação de `leader confirmed: node 4` em todos os nós.

Apesar da redundância de mensagens, ambas as eleições convergem corretamente para o mesmo líder (nó 4, o de maior identificador do anel), confirmando que o algoritmo mantém a corretude mesmo sob concorrência de múltiplas eleições. A duplicação de mensagens, entretanto, evidencia uma ineficiência: o algoritmo, em sua forma básica, não possui mecanismo para detectar e descartar eleições redundantes já resolvidas por outra em andamento.

6.9 Teste 3.3

```
node2 | [node 2 | T=14] forwarding candidate 4
node2 | [node 2 | T=16] received Lamport event from node 1 02:09:04
node3 | [node 3 | T=13] sent Lamport event (wall-clock: 02:09:08)
node2 | [node 2 | T=17] sent Lamport event (wall-clock: 02:09:09)
node4 | [node 4 | T=14] received Lamport event from node 3 02:09:11
node3 | [node 3 | T=16] forwarding candidate 4
node3 | [node 3 | T=18] received Lamport event from node 2 02:09:12
node0 | [node 0 | T=13] sent Lamport event (wall-clock: 02:09:14)
node4 | [node 4 | T=18] I AM THE LEADER (id=4), sending ELECTED
node1 | [node 1 | T=15] sent Lamport event (wall-clock: 02:09:16)
node2 | [node 2 | T=18] sent Lamport event (wall-clock: 02:09:18)
node1 | [node 1 | T=16] received Lamport event from node 0 02:09:20
node3 | [node 3 | T=19] sent Lamport event (wall-clock: 02:09:21)
node2 | [node 2 | T=19] received Lamport event from node 1 02:09:22
node0 | [node 0 | T=14] sent Lamport event (wall-clock: 02:09:23)
node1 | [node 1 | T=17] sent Lamport event (wall-clock: 02:09:25)
node3 | [node 3 | T=20] received Lamport event from node 2 02:09:25
node2 | [node 2 | T=20] received Lamport event from node 1 02:09:26
^M^[Knode4 exited with code 137
```

Fig 10 – Funcionamento do algoritmo de eleição com nó 4 parado depois de se confirmar como líder.

O trecho de log evidencia o comportamento do sistema quando o nó vencedor da eleição falha antes de anunciar sua liderança aos demais. O nó 4 reconhece que é o processo de maior identificador do anel (I AM THE LEADER (id=4), sending ELECTED), porém é encerrado (node4 exited with code 137) antes de efetivamente transmitir a mensagem ELECTED pela rede.

Como resultado, nenhum outro nó do anel recebe a confirmação do novo líder — a mensagem `leader confirmed` não aparece em nenhum log subsequente. Isso evidencia uma limitação crítica do algoritmo: mesmo tendo determinado corretamente o vencedor da eleição, a ausência de um mecanismo de confirmação com timeout ou retransmissão faz com que o anel permaneça indefinidamente sem coordenador confirmado, exigindo uma nova eleição para restabelecer a liderança.

6.10 Teste 3.4

```
106 node0 | [node 0 | T=1] started election with candidate 0
107 node1 | [node 1] connected to node2:6000
108 node2 | [Listen] previous node connected
109 node1 | [node 1 | T=1] started election with candidate 1
110 node1 | [node 1 | T=2] forwarding candidate 1
111 node0 | [Listen] previous node connected
112 node4 | [node 4] connected to node0:6000
113 node2 | [node 2 | T=1] sent Lamport event (wall-clock: 02:00:40)
114 node0 | [node 0 | T=2] sent Lamport event (wall-clock: 02:00:40)
115 node1 | [node 1 | T=4] sent Lamport event (wall-clock: 02:00:40)
116 node4 | [node 4 | T=1] sent Lamport event (wall-clock: 02:00:41)
117 ^M^[[Node4 exited with code 137
118 node3 | [node 3 | T=2] received Lamport event from node 2 02:00:43
119 node2 | [node 2 | T=2] forwarding candidate 2
120 node2 | [node 2 | T=4] forwarding candidate 2
121 node2 | [node 2 | T=6] received Lamport event from node 1 02:00:44
122 node0 | [node 0 | T=3] received Lamport event from node 4 02:00:44
123 node1 | [node 1 | T=5] received Lamport event from node 0 02:00:44
124 node3 | [node 3 | T=3] sent Lamport event (wall-clock: 02:00:48)
125 node2 | [node 2 | T=7] sent Lamport event (wall-clock: 02:00:49)
126 node1 | [node 1 | T=6] sent Lamport event (wall-clock: 02:00:49)
127 node3 | [node 3 | T=4] forwarding candidate 3
128 node3 | [node 3 | T=6] forwarding candidate 3
129 node3 | [node 3 | T=8] received Lamport event from node 2 02:00:52
130 node2 | [node 2 | T=8] received Lamport event from node 1 02:00:53
131 node3 | [node 3 | T=9] sent Lamport event (wall-clock: 02:00:57)
132 node1 | [node 1 | T=7] sent Lamport event (wall-clock: 02:00:58)
133 node2 | [node 2 | T=9] received Lamport event from node 1 02:01:01
134 node0 | [node 0 | T=4] sent Lamport event (wall-clock: 02:01:05)
135 node3 | [node 3 | T=10] sent Lamport event (wall-clock: 02:01:06)
```

Fig 10 – Funcionamento do algoritmo de eleição com nó 4 parado antes da passagem dos candidatos.

O trecho de log evidencia quando um nó falha durante a fase de propagação do candidato. As mensagens `ELECTION` circularam normalmente entre os nós 1, 2 e 3, elevando o candidato de 1 para 3 (`forwarding candidate 1 → 2 → 3`). No entanto, o nó 4 — próximo destinatário no anel — foi encerrado (`node4 exited with code 137`) antes de processar e repassar a mensagem. A partir desse ponto, nenhuma nova mensagem `ELECTION` ou `ELECTED` é observada no log: a eleição trava permanentemente, pois a mensagem nunca completa a volta ao redor do anel.

Esse resultado evidencia a mesma limitação estrutural já observada no Token Ring (T2.2/T2.3): o algoritmo em anel, em sua forma clássica, depende da integridade contínua de todos os nós do anel para funcionar, não possuindo mecanismo de reconfiguração automática frente a falhas.

7. Discussão e Limitações

Nenhum dos três algoritmos implementados trata falha de nó (fail-stop) nativamente, como evidenciado pelos testes T2.2, T2.3, T3.3 e T3.4. Em um anel físico ou lógico, a queda de qualquer processo interrompe a circulação de mensagens, travando tanto a exclusão mútua quanto a eleição. Soluções reais mitigam esse problema com mecanismos de heartbeat e timeout para detectar falhas e disparar reconfiguração do anel ou nova eleição, conforme discutido em aula. Além disso, o relógio de Lamport, por definição, não captura concorrência real entre eventos (T1.3), para isso seriam necessários vetores de relógio.

8. Conclusão

A implementação revelou problemas que a leitura do código não deixava evidentes. A primeira versão da eleição em anel descartava candidatos de menor identificador em vez de substituí-los pelo próprio ID antes de repassar, o que travava a eleição sem que fosse óbvio olhando apenas a lógica isolada. Da mesma forma, a versão inicial do Token Ring manteve uma fila de pedidos local a cada nó, o que fazia apenas um nó conseguir entrar na seção crítica. Ambos os erros só ficaram mais evidentes observando os logs de execução real entre processos, reforçando a importância de testar em condições próximas às de produção.

A configuração com Docker mostrou a necessidade de copiar todos os módulos para a imagem (não só o arquivo principal), o comportamento de buffer de saída do Python em containers (`PYTHONUNBUFFERED=1`) e a diferença entre parar um único container e derrubar o ambiente inteiro.

Os testes de falha (T2.2, T2.3, T3.3, T3.4) confirmaram uma limitação estrutural comum aos dois algoritmos: nenhum deles tolera a queda de um nó, pois dependem da integridade contínua do anel para funcionar. Com mais tempo, o próximo passo natural seria adicionar detecção de falhas via heartbeat/timeout, permitindo reconfiguração automática do anel e disparo de nova eleição sem intervenção manual, mas fora.

9. Referências

1. CHAKRABORTY, Anol. Lamport-Clock-Simulator. GitHub, 2023. Disponível em: <https://github.com/AnolChakraborty/Lamport-Clock-Simulator/blob/main/main.py>.
2. BHATTACHARYA, Priyam. java_token_based_DME_algorithm_ring_topology_MSc_AOS_Sem_2. GitHub. Disponível em: https://github.com/itspriyambhattacharya/java_token_based_DME_algorithm_ring_topology_MSc_AOS_Sem_2/blob/main/Process.java. Acesso em: 5 jul. 2026.
3. PRAGO2001. LP-1: DS/Ring_Bully/ring.py. GitHub. Disponível em: https://github.com/Prago2001/LP-1/blob/main/DS/Ring_Bully/ring.py.
4. GEEKSFORGEES. Socket Programming with Multi-threading in Python. Disponível em: <https://www.geeksforgeeks.org/python/socket-programming-multi-threading-python/>.
5. HARRY. Answer to "How to send JSON data over a socket in Python". Stack Overflow, 2020. Disponível em: <https://stackoverflow.com/a/61350221>.
6. JAVIER S. Question: "Connect to a python socket inside a docker-compose mediated docker from host". Stack Overflow, 2021. Disponível em: <https://stackoverflow.com/q/69120262>.